CURSO DE DESARROLLO SEGURO CON OWASP 2025

Manual de Configuración de Comunicación Cifrada (SSL/TLS) entre PostgreSQL 16 e IBM Open Liberty

Protección de datos en tránsito mediante autenticación de servidor con certificados X.509 y validación estricta (verify-full)

> **Alineación con OWASP Top 10 (2025) — Protección de Datos en Tránsito**
> El tráfico sin cifrar entre el servidor de aplicaciones y el motor de base de datos expone credenciales, tokens de sesión y datos sensibles a ataques de intercepción (MitM) y sniffing en la red interna. Habilitar TLS con validación estricta del certificado y del nombre de host (sslMode="verify-full") mitiga fallas criptográficas y fortalece la defensa en profundidad de la arquitectura.

Procedimiento de Implementación: Certificados y Cifrado TLS en PostgreSQL 16

PASO 1 (a) | Descargar e instalar OpenSSL

Descarga e instala OpenSSL para Windows y asegúrate de que el ejecutable openssl se encuentre agregado a la variable de entorno PATH del sistema para poder invocarlo desde PowerShell.

PASO 2 (b) | Crear el directorio para alojar la llave privada y el certificado

Abre una consola de PowerShell y ejecuta los siguientes comandos para crear y acceder al directorio dedicado a los certificados:

```powershell
mkdir C:\certificados-postgres
cd C:\certificados-postgres
```

PASO 3 (c) | Generar la llave privada RSA

Dentro del directorio creado, genera una llave privada RSA de 2048 bits mediante el siguiente comando:

```powershell
openssl genrsa -out postgresql.key 2048
```

PASO 4 (d) | Generar el certificado autofirmado X.509 con SAN

Genera el certificado público a partir de la llave privada creada anteriormente. Se incluyen localhost, 127.0.0.1 y 192.168.1.64 como nombres alternativos del sujeto (Subject Alternative Name - SAN) para validar la identidad del servidor al conectarse mediante cualquiera de esas direcciones:

```powershell
openssl req -new -x509 `
    -key postgresql.key `
    -out postgresql.crt `
    -days 365 `
    -sha256 `
    -subj "/CN=localhost" `
    -addext "subjectAltName=DNS:localhost,IP:127.0.0.1,IP:192.168.1.64"
```

> **Importante sobre el atributo SAN (Subject Alternative Name)**
> El campo SAN debe incluir exactamente el nombre de dominio o la dirección IP que utilices en el atributo serverName del DataSource. Si PostgreSQL se ejecuta en otro servidor, sustituye o agrega su dirección IP o FQDN real antes de emitir el certificado.

PASO 5 (e) | Localizar los archivos de configuración de PostgreSQL

Conéctate a tu instancia de PostgreSQL (mediante psql o pgAdmin) y ejecuta las siguientes consultas para identificar las rutas exactas de los archivos de configuración:

```sql
SHOW config_file;
SHOW hba_file;
SHOW data_directory;
```

PASO 6 (f) | Habilitar SSL en postgresql.conf

Abre el archivo postgresql.conf obtenido en el paso anterior y agrega o actualiza las siguientes directivas:

```ini
ssl = on

ssl_cert_file = 'C:/certificados-postgres/postgresql.crt'

ssl_key_file = 'C:/certificados-postgres/postgresql.key'
```

> **Control de permisos en Windows (Principio de Mínimo Privilegio)**
> Verifica que las rutas existan en el equipo donde se ejecuta PostgreSQL. En entornos Windows, la cuenta de servicio

> bajo la cual corre PostgreSQL debe tener permisos de lectura sobre ambos archivos, especialmente sobre la llave privada (postgresql.key). Restringe los permisos NTFS de la llave privada para impedir que otros usuarios locales puedan leerla.

PASO 7 (g) | Exigir conexiones cifradas en pg_hba.conf

Por defecto, PostgreSQL puede aceptar conexiones cifradas y sin cifrar simultáneamente. Para obligar el uso de TLS en nuestra aplicación y rechazar cualquier intento en texto plano, agrega las siguientes reglas al inicio de pg_hba.conf (antes de las reglas genéricas existentes) cuando Open Liberty y PostgreSQL residan en el mismo equipo:

```CONF
# Rechazar conexiones sin TLS
hostnossl usuarios postgres 127.0.0.1/32 reject
hostnossl usuarios postgres ::1/128 reject

# Permitir conexiones con TLS
hostssl usuarios postgres 127.0.0.1/32 scram-sha-256
hostssl usuarios postgres ::1/128 scram-sha-256
```

Nota: Esta configuración asume que el usuario postgres emplea autenticación SCRAM-SHA-256. Si utilizas otro método de autenticación configurado en tu entorno, conserva el mecanismo correspondiente. Si PostgreSQL está alojado en un servidor remoto, sustituye las direcciones de loopback (127.0.0.1/32 y ::1/128) por la dirección IP del servidor Open Liberty.

PASO 8 (h) | Reiniciar el servicio PostgreSQL y verificar el estado SSL

Reinicia el servicio de PostgreSQL desde el administrador de servicios de Windows (services.msc) o mediante consola para aplicar los cambios. Posteriormente, verifica que el cifrado esté activo ejecutando:

```SQL
SHOW ssl;
```

Resultado esperado: on

PASO 9 (i) | Preparar el certificado público para IBM Open Liberty

El controlador PostgreSQL JDBC permite utilizar directamente el certificado en formato PEM (postgresql.crt) mediante la propiedad sslRootCert, sin necesidad de generar un segundo certificado ni construir obligatoriamente un almacén JKS. Open Liberty expone de forma nativa las propiedades ssl, sslMode y sslRootCert para orígenes de datos PostgreSQL.

Copia o mantén el certificado público en una ruta accesible para el proceso de Open Liberty, por ejemplo:

```TEXT
```

```
C:\certificados-postgres\postgresql.crt
```

PASO 10 (j) | Configurar el DataSource en server.xml

Localiza la definición actual de tu origen de datos en el archivo server.xml de Open Liberty:

```xml
<dataSource id="UsuariosDataSource" jndiName="jdbc/Usuarios">
    <jdbcDriver libraryRef="PostgreSQLLib"/>
    <properties.postgresql serverName="localhost"
                           portNumber="5432"
                           databaseName="usuarios"
                           user="postgres"
                           password="root"/>
</dataSource>
```

Actualiza el bloque agregando las propiedades de seguridad TLS:

```xml
<dataSource id="UsuariosDataSource" jndiName="jdbc/Usuarios">
    <jdbcDriver libraryRef="PostgreSQLLib"/>
    <properties.postgresql
        serverName="localhost"
        portNumber="5432"
        databaseName="usuarios"
        user="postgres"
        password="root"
        ssl="true"
        sslMode="verify-full"
        sslRootCert="C:/certificados-postgres/postgresql.crt"/>
</dataSource>
```

Propiedad	Función de Seguridad
`ssl="true"`	Habilita el cifrado SSL/TLS en la capa de transporte del controlador JDBC.
`sslMode="verify-full"`	Exige conexión cifrada, valida la cadena de confianza del certificado y verifica que el nombre del servidor coincida con el CN o SAN.
`sslRootCert`	Especifica la ruta al certificado raíz o autofirmado confiable que se utilizará como ancla de confianza.

PASO 11 (k) | Verificar la conexión TLS directamente mediante psql

Antes de probar la aplicación empresarial completa, valida el enlace cifrado directamente desde el cliente de línea de comandos psql:

```powershell
psql "host=localhost port=5432 dbname=usuarios user=postgres sslmode=verify-full
sslrootcert=C:/certificados-postgres/postgresql.crt"
```

Una vez establecida la sesión, consulta la vista del sistema pg_stat_ssl para confirmar la versión del protocolo TLS y el algoritmo de cifrado negociado:

```sql
SELECT
    ssl,
    version,
    cipher
FROM pg_stat_ssl
WHERE pid = pg_backend_pid();
```

Si la columna ssl devuelve true (t), el canal de comunicación está operando de forma cifrada.

PASO 12 (l) | Validar la conexión cifrada desde IBM Open Liberty

Inicia el servidor IBM Open Liberty y ejecuta una operación de prueba en tu aplicación. Si la configuración es correcta, se ejecutará el siguiente flujo de validación:

1. Open Liberty inicia la negociación TLS con PostgreSQL 16.
2. PostgreSQL presenta su certificado digital (postgresql.crt).
3. El controlador JDBC verifica que el certificado esté firmado por una entidad confiable configurada en sslRootCert.
4. El controlador comprueba que el nombre del servidor (serverName) coincida con el atributo SAN del certificado.
5. Se establece el pool de conexiones JDBC cifradas.

> **Comportamiento esperado ante certificados inválidos (Fail-Secure)**
> Si el certificado ha expirado, no coincide con sslRootCert o el nombre de host no figura en el SAN, la conexión será rechazada inmediatamente. Este comportamiento fail-secure es el objetivo principal del modo verify-full, evitando ataques de suplantación o aceptación de certificados no confiables.